Multi-Stage AI Workflow: Child Nutrition Screening

Final Capstone – Multi-Stage AI Workflow Across UX Types · Applied AI & Prompt Engineering
Christine Umutoni · Data Engineering · Repository: https://github.com/mutochris04-ctrl/multi-stage-ai-workflow

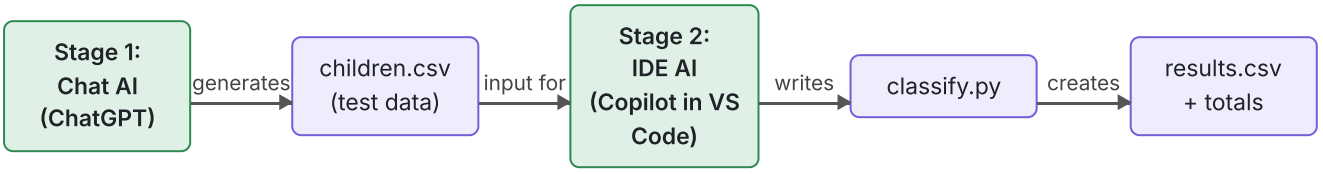
1. Problem

Health workers screen children aged 6–59 months using MUAC (mid-upper arm circumference) and a check for oedema. Classifying each child by hand is slow and easy to get wrong. This workflow chains two AI tools of different UX types: a chat AI creates the test data, and an IDE AI uses that data to write a program that classifies every child automatically.

2. AI tools used (two UX types)

Stage	UX type	Tool I used	Output	Can be replaced by
1	Chat-based AI	ChatGPT	children.csv	Gemini, Claude, Copilot chat
2	IDE-based AI	GitHub Copilot in VS Code	classify.py	Cursor, Gemini Code Assist

3. Workflow diagram



The output of the chat AI (children.csv) becomes the input of the IDE AI.

4. Step-by-step

1. **Chat AI creates the data.** I asked ChatGPT for a CSV of 15 children (child_id, name, sex, age_months, muac_mm), then a follow-up to add an oedema column. Prompts are in section 7.
2. **I saved the output** as `children.csv` in the project folder in VS Code.
3. **IDE AI writes the code.** I gave Copilot in VS Code a prompt that uses `children.csv` as input. It created `classify.py`.
4. **I checked the code** to make sure it uses simple Python (one function, a for loop, if/elif/else, lists) and the correct rules.
5. **I ran the program** with the Run button. It printed each child's status and the totals, and saved `results.csv`.
6. **I uploaded the project to GitHub** in separate commits: Stage 1 data, Stage 2 code and output, then documentation.

5. Classification rules (CMAM)

Condition	Status
Oedema = yes (any MUAC)	Severe (SAM)
MUAC < 115 mm	Severe (SAM)
MUAC 115–124 mm	Moderate (MAM)
MUAC ≥ 125 mm	Normal

6. Results and testing

Severe: 7 · Moderate: 3 · Normal: 5 (checked by hand against the rules).

- Jean, MUAC exactly 115 → Moderate (the boundary works)
- Diane, MUAC 125 with oedema → Severe (the oedema rule comes first)

7. Prompts used

Stage 1 – Chat AI (ChatGPT):

```
Create a small test dataset in CSV format about children checked for malnutrition.
Columns: child_id, name, sex, age_months, muac_mm
Rules:
- 15 rows
- child_id goes from 1 to 15
- sex is M or F
- age_months is a whole number between 6 and 59
- muac_mm is a whole number between 100 and 140
- include at least one child with muac_mm exactly 115 and one with exactly 125
Give me ONLY the CSV text, with no explanation.
```

Stage 1 – follow-up:

```
Add one more column at the end called oedema, with the value yes or no.
Rules:
- most children have no
- exactly 3 children have yes
- at least one child with yes must have muac_mm of 125 or more
Keep all other values the same. Give me ONLY the CSV text, with no explanation.
```

Stage 2 – IDE AI (Copilot in VS Code):

```
Read the file children.csv in this folder and create a Python file called classify.py.
The program must:
1. Open children.csv and read it line by line (do not use pandas, use only basic Python).
2. For each child, decide the nutrition status with these rules, in this order:
   - if oedema is "yes" -> "Severe (SAM)"
   - else if muac_mm is less than 115 -> "Severe (SAM)"
   - else if muac_mm is less than 125 -> "Moderate (MAM)"
   - else -> "Normal"
3. Print one line per child: name, muac_mm, oedema, status.
4. At the end, print how many children are Severe, Moderate and Normal.
5. Save the results in a new file called results.csv with the columns:
   child_id, name, muac_mm, oedema, status.
Keep the code simple for a beginner: use one function called classify_child,
a for loop, if/elif/else and lists. Add short comments explaining each part.
```

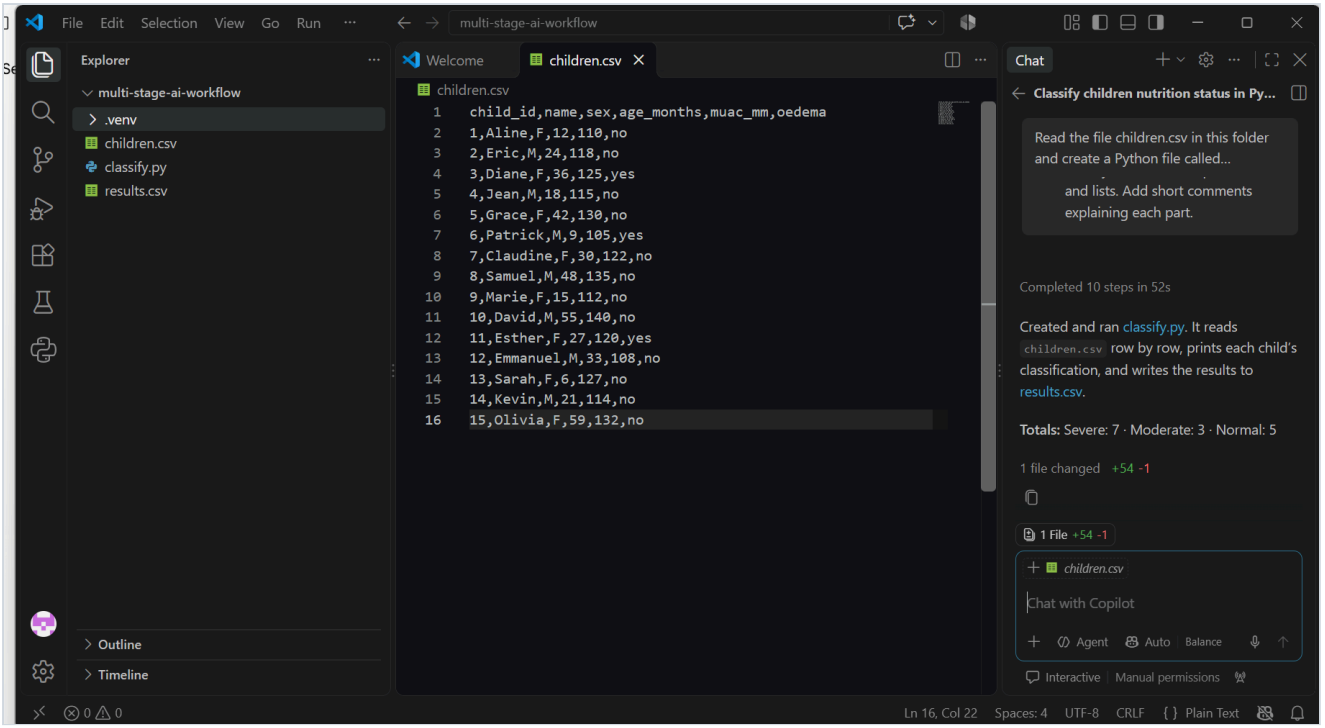
8. Adaptability (tool-independence)

The prompts are plain text and the data is a plain CSV, so any chat AI and any IDE AI can do the same stages. The final program uses only Python's built-in `csv` module, so it runs without any AI tool and without installing anything.

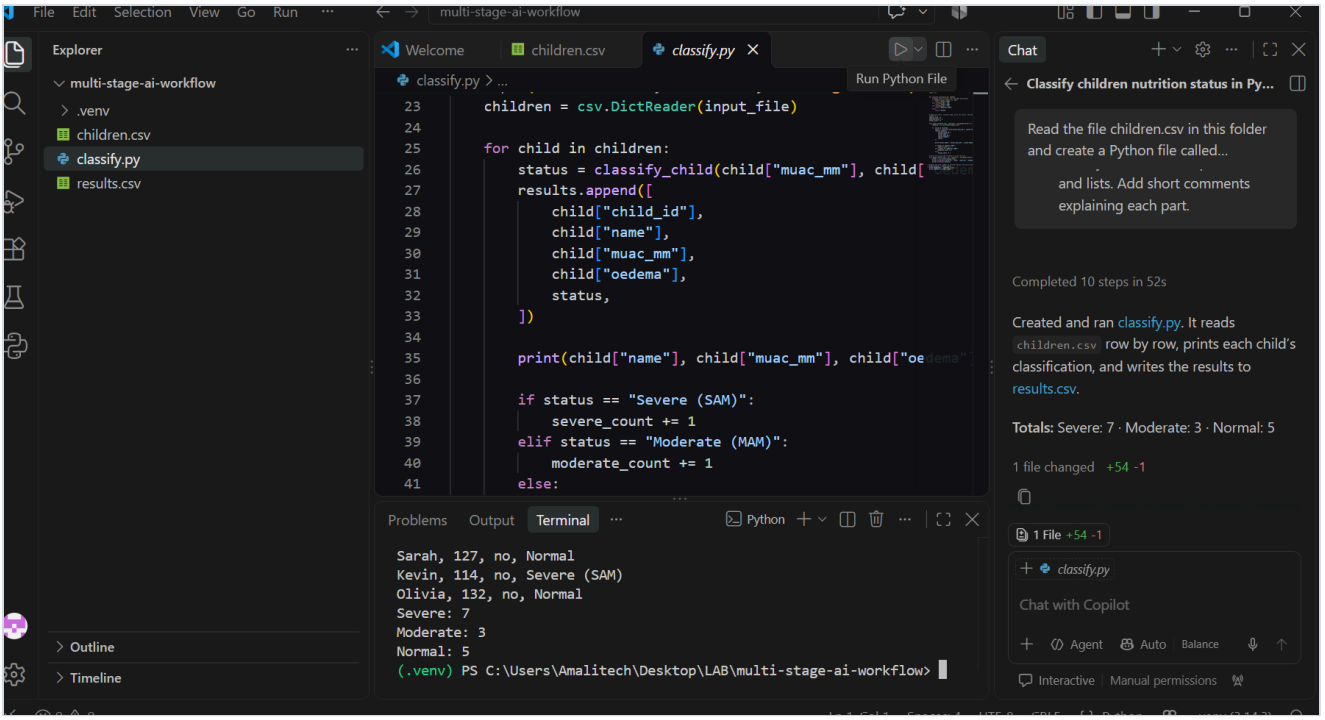
9. Efficiency (how it saves time)

Classifying a list of children by hand means checking each one against the rules. With this workflow, a new list only needs to be saved as `children.csv`, and one click on Run classifies everyone, counts the totals and saves the results.

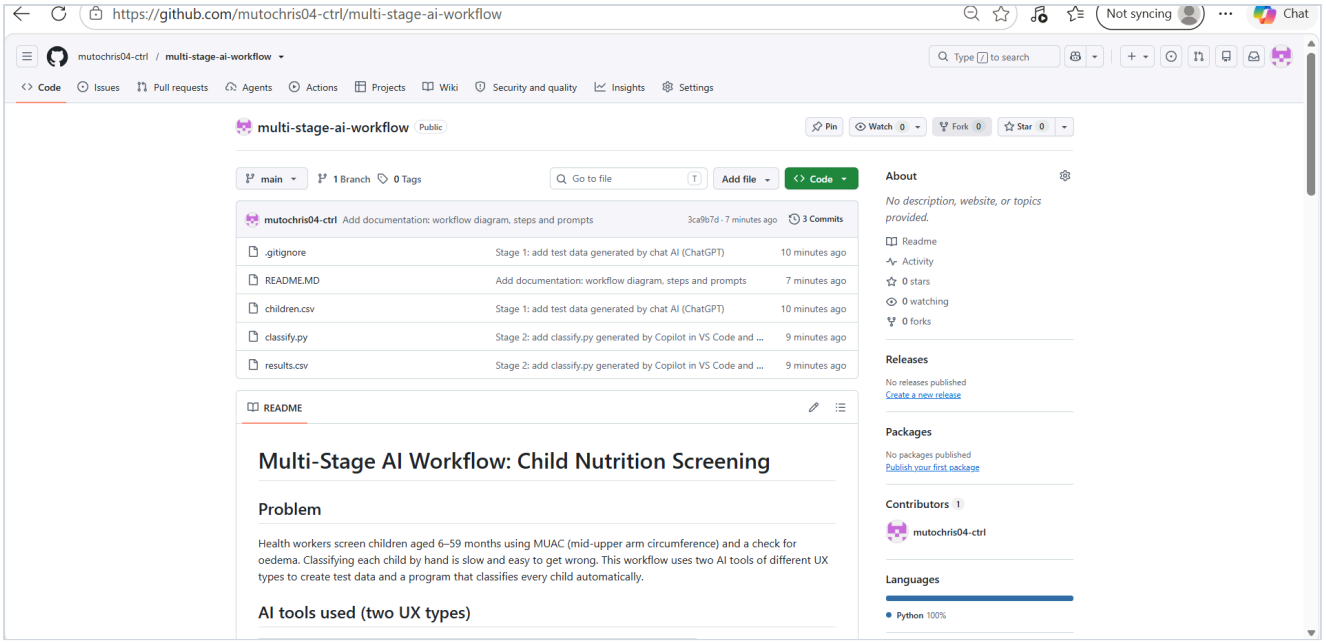
10. Evidence



Stage 2: Copilot in VS Code created and ran classify.py from children.csv.



Running classify.py: each child's status and the totals (Severe 7, Moderate 3, Normal 5).



GitHub repository with the stage-by-stage commits.

11. Note

All data is made up (synthetic) for testing. No real children's data is used.